CONTA COMIGO · ESTUDO DE VIABILIDADE II

Instalação nativa Android, iOS e Windows

Análise técnica para instalar o jogo como aplicativo nativo nos três grandes sistemas sem reescrever o código: iOS pela App Store via Capacitor, com voz nativa e armazenamento persistente no WebKit; TWA Android e Electron Windows mantidos da Fase 1; conformidade infantil nas três lojas e plano de ação em quatro fases no GitHub Actions.

Outubro de 2026 · Base analisada: v2.6.0

Klivfy Tecnologia · Conta Comigo — Matemática para Crianças

Sumário

1. Sumário Executivo 2

2. O Que "Instalação Nativa" Significa nos Três Sistemas 2

3. iOS — O Território Novo 3

3.1 App Store, Kids Category e TestFlight . . . 3

3.2 A diretriz 4.2 (funcionalidade mínima) e a defesa do jogo . . . 3

3.3 Web apps da Tela de Início — o atalho provisório . . . 4

4. Fatores Críticos no iOS/WebKit — Voz e Armazenamento 4

4.1 Voz: a API web funciona, mas o iOS tem particularidades . . . 4

4.2 Armazenamento: localStorage transiente em WKWebView . . . 5

4.3 Ícone, splash, safe-area e apresentação . . . 5

5. Android e Windows — Reaproveitar o Que Já Existe 6

6. Rotas iOS Comparadas 7

7. Conformidade Infantil nas Três Lojas 8

8. Custos, Contas e Cronograma 9

9. Veredito e Esforço por Rota 9

10. Plano de Ação em Fases 10

11. Riscos e Mitigações 11

12. Conclusão 12

1. Sumário Executivo

A nova demanda da Klivfy Tecnologia amplia o alcance do pedido anterior: em vez de apenas Android e Windows, o Conta Comigo deve poder ser **instalado como aplicativo nativo nos três grandes sistemas operacionais** — Android, iOS e Windows 10/11 — a partir de um caminho distinto da distribuição web atual. Este estudo II avalia a viabilidade técnica dessa extensão e conclui, com margem confortável, que ela é **viável para os três sistemas sem reescrever o jogo**. O motor pedagógico validado até a v2.6.0 continua sendo a única base de código; as plataformas nativas passam a ser cascos de instalação e distribuição em volta dele.

A peça genuinamente nova é o **iOS**, que não tinha sido coberto pelo estudo anterior e é o caso mais restritivo dos três: não existe TWA no ecossistema Apple, não existe sideload oficial e o único canal de distribuição em escala é a App Store, exigindo conta do Apple Developer Program (US$ 99 por ano) e aprovação da revisão da Apple. A rota recomendada para o iOS é o **Capacitor**, que gera um projeto iOS real (Swift/Xcode) executando o jogo em WKWebView, com duas adaptações pequenas e já mapeadas no motor: um segundo back-end de voz (plugin nativo de síntese, pois a voz web no iOS tem particularidades documentadas no capítulo 4) e um adaptador de armazenamento persistente (a API Preferences, pois o localStorage em WKWebView pode ser recolhido pelo sistema).

Android e Windows não exigem trabalho novo imediato: a TWA e o Electron já foram implementados na Fase 1 e compilam no GitHub Actions. O estudo analisa, contudo, a alternativa de **unificar o celular em um único Capacitor** (Android e iOS) e recomenda **manter a pilha dupla** — TWA no Android, Capacitor só no iOS — porque a TWA preserva a voz plena do Chrome, pesa 2 a 4 MB e permite atualizar o jogo sem reenviar pacote à loja, enquanto a rota Capacitor Android obrigaria plugin de TTS, pacote maior e nova revisão a cada atualização. O custo adicional do plano de três lojas é concentrado numa única despesa nova: a taxa anual da Apple; o restante permanece próximo de zero.

100% do jogo aproveitado — a expansão para iOS não reescreve nenhuma linha do gameplay

Android: VIÁVEL (TWA em produção) · iOS: VIÁVEL (Capacitor + adaptações mapeadas) · Windows: VIÁVEL (Electron em produção)

2. O Que "Instalação Nativa" Significa nos Três Sistemas

Antes de comparar rotas, é preciso fixar o que cada sistema entende por instalação de aplicativo, porque os três ecossistemas impõem regras muito diferentes. No **Android**, um aplicativo é um pacote APK/AAB instalado pela Play Store ou diretamente pelo arquivo (sideload, permitido de fábrica com confirmação do usuário). No **Windows 10/11**, a instalação nativa é um executável com instalador (NSIS/MSIX), distribuído pela Microsoft Store ou pelo download direto — sem qualquer gate corporativo. Já no **iOS**, a regra é única e rígida: o iPhone só executa aplicativos assinados pela Apple, distribuídos pela App Store (ou TestFlight, para testes), e não há sideload oficial — em nenhuma versão atual, incluindo o iOS 26.

Essa assimetria define a arquitetura da resposta. Para Android e Windows, o caminho distinto que o pedido descreve — "instalação nativa" — já está materializado pela Fase 1: a TWA é um pacote Android de verdade

publicado na Play, e o Electron é um executável Windows de verdade. Para o iOS, "instalação nativa" significa, obrigatoriamente, **passar pela App Store** com um projeto Xcode assinado — daí a escolha do Capacitor, que gera exatamente esse projeto a partir da mesma base web. Existe ainda no iOS a pseudo-instalação por "web app da Tela de Início" (ícone na tela inicial que abre o jogo em tela cheia, padrão desde o iOS 26), útil como atalho provisório e sem custo, mas que não é um aplicativo da App Store e herda as limitações do Safari — ela é analisada como rota complementar no capítulo 6.

Aspecto	Android	iOS	Windows 10/11
Formato do pacote	APK / AAB	IPA (Xcode, assinado)	EXE (NSIS) / MSIX
Canal oficial	Play Store	App Store (única em escala)	Microsoft Store + download direto
Sideload oficial	Sim (com confirmação)	Não (TestFlight para testes)	Sim (download direto)
Wrapper web oficial	TWA (Chrome completo)	Não existe equivalente	WebView2 / Electron
Taxa de desenvolvedor	US$ 25 (única)	US$ 99 / ano	R$ 0 (individual, fluxo 2026)
Status no projeto	TWA implementada (Fase 1)	A criar — rota Capacitor	Electron implementado (Fase 1)

Tabela 1 — O que significa instalar um aplicativo em cada sistema (estado atual, 2026).

3. iOS — O Território Novo

3.1 App Store, Kids Category e TestFlight

Publicar no iOS significa subir um IPA para a App Review dentro de um programa pago: o **Apple Developer Program custa US$ 99 por ano**, com renovação anual, e inclui TestFlight (distribuição beta para até 10 mil testadores), App Store Connect e os certificados de assinatura. É a única despesa recorrente introduzida por esta expansão — o Google Play cobra US$ 25 uma única vez, e a Microsoft Store, no fluxo atual para desenvolvedores individuais, não cobra registro. Para um jogo infantil, a Apple oferece a **Kids Category** (faixas de 5 anos ou menos, 6-8 e 9-11 — o Conta Comigo se encaixa na primeira), cuja diretriz 1.3 proíbe links para fora do app, compras e distrações fora de um portão parental, além de **proibir analytics e publicidade de terceiros**.

Esse conjunto de exigências, que derruba muitos aplicativos concorrentes, é exatamente o que o Conta Comigo já pratica por construção desde a origem: zero coleta, zero telemetria, zero anúncios, zero SDKs de terceiros, portão dos responsáveis implementado desde a v2.3 e política de privacidade publicada em URL própria. Em 2025 a Apple ainda reforçou o aparato de idade (API Declared Age Range, em que responsáveis compartilham faixas etárias em vez de datas de nascimento), reforçando justamente o modelo sem coleta que o jogo adota. A expectativa razoável, portanto, é de revisão tranquila — com o formulário de etiquetas de privacidade respondendo "não coletamos dados", a variante mais simples possível.

3.2 A diretriz 4.2 (funcionalidade mínima) e a defesa do jogo

O risco de revisão mais citado para aplicativos que embrulham conteúdo web é a diretriz **4.2 — Minimum Functionality**, usada pela Apple para rejeitar "sites empacotados" sem valor agregado de aplicativo. A defesa do Conta Comigo é estrutural, e não cosmética: o jogo **roda integralmente no aparelho**, com motor pedagógico próprio (geração de questões por nível, duelos, conquistas, loja, mascotes), persistência local no aparelho, funcionamento offline total e nenhuma página web espelhada — não há um "site do jogo" que o app apenas exiba. Além disso, a versão nativa iOS acrescentará capacidades genuínas de plataforma: síntese de voz pelo sintetizador nativo do sistema (AVSpeechSynthesizer), armazenamento persistente pela API nativa, ícone, splash e comportamento standalone.

A literatura de rejeições da 4.2 concentra-se em apps que apenas carregam um endereço remoto em WebView. Aqui o binário carrega o jogo inteiro embutido (os assets vão dentro do IPA), o que o aproxima muito mais de um jogo offline tradicional do que de um wrapper. Como mitigação operacional, o plano usa o **TestFlight antes da submissão**: a mesma build é validada por famílias reais, e os metadados da loja descrevem o jogo como aplicativo educativo offline — nunca como "versão web". Caso a revisão peça ajustes, o ciclo TestFlight-correção é curto e não bloqueia as demais plataformas.

3.3 Web apps da Tela de Início — o atalho provisório

Desde o iOS 16.4 os web apps instalados ganharam notificações push, e o iOS 26 tornou o comportamento padrão: **todo site adicionado à Tela de Início abre como web app** em tela cheia, com ícone próprio. A Apple chegou a anunciar a remoção desse recurso na União Europeia sob a DMA (início de 2024) e **reverteu a decisão semanas depois**, mantendo o suporte. Para o Conta Comigo, esse atalho custa dois dias (a PWA do GitHub Pages já é instalável) e zero recorrente, mas fica fora da App Store, depende do Safari e herda as restrições de voz e armazenamento descritas no próximo capítulo — por isso é tratado como rota provisória de alcance, não como resposta ao pedido.

4. Fatores Críticos no iOS/WebKit — Voz e Armazenamento

Dois fatores técnicos concentram praticamente todo o risco da rota iOS, e ambos têm mitigação conhecida, barata e já planejada como código do motor. O princípio do estudo I continua valendo e se intensifica no iOS: para um público pré-leitor de 4 anos, a narração é a interface; e o progresso acumulado (36 figurinhas, 20 conquistas, perfis) não pode evaporar. O capítulo seguinte descreve cada fator, o comportamento real observado na plataforma e a adaptação correspondente no engine.js.

4.1 Voz: a API web funciona, mas o iOS tem particularidades

Ao contrário do WebView Android (onde a API não existe — achado crítico do estudo I), o **iOS expõe speechSynthesis no Safari e no WKWebView desde 2018**, e é plataforma de referência da API no MDN. Porém, a experiência documentada pela comunidade mostra particularidades que o motor precisa tratar: a primeira fala deve ocorrer após um gesto do usuário (padrão que o jogo já segue, pois toda narração responde a um toque); em versões do iOS a fila de fala **silencia chamadas se não houver cancel() prévio** — a receita comunitária é cancelar antes de cada speak(); a **lista de vozes varia entre versões** (o iOS 18 removeu vozes que o 18.2 restituiu; o iOS 17 falhou com certos textos); e as vozes naturais em português (família Luciana/Francisca) dependem de estarem instaladas no aparelho.

A mitigação é estrutural e definitiva: a v2.4.0 já concentra toda a fala num único ponto de decisão, então o plano adiciona ao motor um **segundo back-end de voz** acionado quando o jogo roda dentro do Capacitor — o plugin nativo de síntese (família capacitor-speech-synthesis / Text-to-Speech), que usa o AVSpeechSynthesizer do iOS e o TextToSpeech do Android, com as vozes do sistema e comportamento estável entre versões. A detecção é automática em tempo de execução: no Chrome/Edge/Firefox/Electron usa a API web como hoje; no Capacitor, o plugin nativo; se o plugin faltar, cai na web com as proteções do parágrafo anterior. Esforço estimado: 2 a 3 dias de adaptação, cobertos pela Fase A.

4.2 Armazenamento: localStorage transiente em WKWebView

A documentação oficial do Capacitor é explícita: no iOS, o **localStorage e o IndexedDB de um WKWebView devem ser considerados transitórios**, porque o sistema pode recolher esse armazenamento quando o aparelho estiver com pouco espaço; no Android vale regra parecida para o WKWebView, embora a API de persistência do navegador permita marcar dados como persistentes. Para um jogo cujo progresso é o ativo mais valioso da família, isso não é aceitável como única camada.

A mitigação é o **adaptador de armazenamento**: quando o jogo detecta o ambiente Capacitor, grava por meio da API **Preferences** (armazenamento nativo do app, fora do alcance da evicção de WebView), mantendo o localStorage como fallback nas demais plataformas — a arquitetura de persistência já é abstrata (chrome.storage no Chrome, localStorage na web), então o adaptador entra na mesma costura. Como cinta de segurança, o backup de um clique do painel dos responsáveis (v2.5) continua disponível para a família exportar o progresso a qualquer momento. Esforço estimado: 2 dias.

4.3 Ícone, splash, safe-area e apresentação

Os demais requisitos de apresentação do iOS são trabalho de configuração, não de pesquisa: o Capacitor gera os tamanhos de ícone e as telas de splash a partir dos assets existentes (a PWA já tem ícones 192/512 e maskable), e a v2.6.0 já trata áreas seguras (viewport-fit=cover, dvh e safe-area) e funciona em modo standalone — exatamente o modo em que o WKWebView do Capacitor executa o jogo. A orientação "qualquer" do manifesto é respeitada. Não há fator crítico conhecido nesta camada; a validação prática acontece na Fase B, em aparelho real.

Fator	Comportamento no iOS	Mitigação no motor	Esforço
Voz (speechSynthesis)	Disponível desde 2018, com particularidades: fila silencia sem cancel(); vozes variam por versão (17/18); gesto obrigatório	cancel() antes de speak(); 2º back-end nativo (AVSpeechSynthesizer) via plugin, com detecção automática	2-3 dias
Armazenamento	localStorage/IndexedDB de WKWebView são transitórios (evicção sob pressão de espaço)	Adaptador para Preferences API (nativo, sem evicção); backup 1-clique como cinta	2 dias
Apresentação	Ícone, splash e standalone exigem projeto Xcode	Capacitor gera a partir dos assets existentes; safe-area já tratada na v2.6.0	Configuração
Assinatura/distribuição	Só Apple assina; App Store/TestFlight; conta anual	Apple Developer Program US$ 99/ano; secrets no CI; TestFlight para beta	Administrativo

Tabela 2 — Fatores críticos do iOS/WebKit e mitigações correspondentes.

5. Android e Windows — Reaproveitar o Que Já Existe

A Fase 1, executada nesta mesma série de entregas, deixou Android e Windows cobertos: a **TWA compila no GitHub Actions e gera APK/AAB assinados** com verificação automática do assetlinks.json, e o **Electron gera instalador NSIS e versão portable** pelo mesmo pipeline. Nenhum dos dois precisa ser refeito para atender ao pedido de instalação nativa — eles já são pacotes nativos nos respectivos sistemas. A pergunta que resta é arquitetural: vale a pena **unificar o celular num único Capacitor** (um só projeto Android+iOS), abandonando a TWA, para reduzir a variedade de ferramentas? A resposta recomendada é não, por três razões objetivas.

Primeiro, a **voz**: na TWA o jogo roda no Chrome completo, com o sintetizador do Google e as vozes que a família já conhece; no Capacitor Android, a API web não existe no WebView (issue 40417848 do Chromium) e a voz passaria a depender do plugin nativo — funcional, mas com outro sintetizador e outro comportamento. Segundo, a **atualização**: com a TWA, o jogo servido pelo GitHub Pages é atualizado num push; com o Capacitor os assets ficam embutidos no APK, exigindo novo pacote e nova revisão do Play a cada mudança — no iOS essa passagem pela revisão é inevitável (não há alternativa), no Android ela é evitável e vale evitá-la. Terceiro, o **peso e o esforço**: 2-4 MB e zero dias contra 8-15 MB e cerca de 12 dias. A TWA permanece como rota Android; o Capacitor entra apenas onde é necessário, no iOS.

Critério	TWA (mantida) + Capacitor iOS	Capacitor unificado (Android + iOS)
Voz no Android	Plena — Chrome completo (rota atual)	Plugin nativo (outro sintetizador)
Atualização do jogo no Android	Push no Pages, sem nova revisão	Novo APK/AAB + revisão do Play
Ferramentas de mobile	2 (Bubblewrap + Capacitor)	1 (só Capacitor)
Esforço incremental	Zero dias no Android (feito)	cerca de 12 dias
Peso no Android	2-4 MB	8-15 MB
Risco	Baixo (rota validada)	Médio (voz e revisão por update)

Tabela 3 — Pilha dupla (recomendada) versus unificação mobile em Capacitor.

6. Rotas iOS Comparadas

Quatro rotas foram avaliadas para o iOS, com o mesmo critério de desempate da série: preservar a voz e o progresso da criança, minimizar reescrita e encaixar no pipeline de CI já existente. A **rota recomendada é o Capacitor com o plugin nativo de TTS**: gera um projeto iOS real (o que a App Store exige), embute o jogo no IPA, e as duas adaptações do motor (voz e armazenamento) são pequenas e beneficiam todas as plataformas. O **web app da Tela de Início** entra como complemento de custo zero: em dois dias dá para instruir famílias iOS a instalar o atalho pela PWA publicada, útil enquanto a conta Apple e a revisão não acontecem — mas não atende ao pedido de instalação nativa, pois não é um app da loja e herda as restrições do Safari.

As reescritas completas repetem o diagnóstico dos estudos anteriores e do capítulo 5: **Flutter** ou **React Native** exigiriam reimplementar o motor pedagógico inteiro (40 a 80 dias) para produzir o mesmo comportamento, com risco de divergência permanente entre versões; **Cordova**, embora tecnicamente capaz de gerar o projeto iOS, é geração legada com comunidade em declínio, sem vantagem sobre o Capacitor (que é, na prática, seu sucessor mantido pela mesma equipe do Ionic). Nenhuma funcionalidade pedida justifica qualquer uma dessas rotas.

Critério	Capacitor + TTS	PWA Tela de Início	Flutter/React Native	Cordova
Esforço	8-12 dias	2 dias	40-80 dias	15-25 dias
App Store	Sim	Não (fora da loja)	Sim	Sim
Voz	Nativa (AVSpeech) + fallback web	Web com particularidades	Nativa	Plugin legado
Progresso persistente	Preferences nativo	Sujeito a evicção	Nativo	Plugin
Custo recorrente	US$ 99/ano	R$ 0	US$ 99/ano	US$ 99/ano
Maturidade	Alta (Ionic/Capacitor)	Padrão Apple	Alta	Em declínio
Veredito	Recomendada	Complemento provisório	Descartada	Descartada

Tabela 4 — Matriz comparativa das rotas iOS.

7. Conformidade Infantil nas Três Lojas

A disciplina arquitetural do projeto — zero coleta, zero telemetria, zero anúncios, zero SDKs de terceiros, portão adulto no painel dos responsáveis e política de privacidade publicada — continua sendo a maior vantagem regulatória do plano, agora em três audits simultâneos. No **Google Play**, o programa Families e o formulário Data Safety recebem a resposta mais simples possível (nenhum dado coletado, nenhuma SDK). Na **App Store**, a Kids Category da diretriz 1.3 proíbe exatamente o que o jogo não tem: analytics e publicidade de terceiros, links para fora e compras fora de portão parental; as etiquetas de privacidade (Privacy Nutrition Labels) respondem "dados não coletados", e a Declared Age Range da Apple reforça o modelo sem coleta de dados de crianças. Na **Microsoft Store**, a certificação pede a mesma tríade: declaração de privacidade, público honesto e ausência de coleta invisível.

Vale destacar o único detalhe operacional novo que a Kids Category exige e o jogo precisa demonstrar na revisão: **portão parental** em qualquer área que leve para fora da experiência infantil. O painel dos responsáveis já é protegido por verificação adulta desde a v2.3, e a versão iOS não terá links externos no fluxo da criança — a página de privacidade vive na web, acessada pelos responsáveis pelos metadados da loja, não por botão dentro da tela do jogo. O classificatório etário sai do questionário IARC (Play), do questionário da Apple (4+, com Kids Category 5 anos ou menos) e do questionário da Microsoft — todas as respostas derivam da mesma ficha pedagógica já escrita.

Requisito	Google Play	App Store (iOS)	Microsoft Store
Conta de desenvolvedor	US$ 25 (única)	US$ 99 / ano	R$ 0 individual; US$ 19 organização
Programa infantil	Families Policy	Kids Category (diretriz 1.3)	Certificação padrão
Faixa etária declarada	IARC: 3+/4+ (Livre)	4+ (Kids: 5 anos ou menos)	IARC: 3+/4+ (Livre)
Analytics/anúncios de 3º	Proibidos (Families)	Proibidos (1.3)	Restritos na certificação
Formulário de dados	Data Safety: nenhum	Labels: não coletamos	Declaração de privacidade
Portão parental	Painel protegido (v2.3)	Exigido — já implementado	Recomendado — já implementado
Política de privacidade	URL já publicada	URL obrigatória (a mesma)	URL obrigatória (a mesma)

Tabela 5 — Requisitos infantis das três lojas frente ao estado atual do jogo.

8. Custos, Contas e Cronograma

O custo do plano de três lojas permanece concentrado numa única despesa nova: a **taxa anual da Apple**. Todo o resto segue gratuito para repositório público — hospedagem no GitHub Pages, compilação no GitHub Actions (os executores macOS, necessários para produzir o IPA, são gratuitos em repositórios públicos), keystore Android já versionado e conta Microsoft sem taxa para desenvolvedores individuais. Somando as três lojas, o primeiro ano custa aproximadamente **US$ 124 (cerca de R$ 650)**: US$ 25 únicos do Play mais US$ 99 da Apple; do segundo ano em diante, apenas a renovação anual da Apple. A distribuição fora de loja — APK direto para famílias Android e instalador Windows — segue sem qualquer custo ou conta.

Item	Custo	Papel no plano
GitHub Pages + Actions (repo público)	R$ 0	Hospedagem da PWA e compilação de todos os pacotes
Executores macOS (build do IPA)	R$ 0 em repo público	Compila o projeto iOS no CI, sem Mac local
Conta Google Play	US$ 25 (única)	Publicar a TWA na Play Store
Apple Developer Program	US$ 99 / ano	TestFlight + App Store (única despesa nova e recorrente)
Conta Microsoft Store (individual)	R$ 0 no fluxo 2026	Publicar o Electron na Store
Certificado de código Windows (opcional)	US$ 100-400 / ano	Remove aviso do SmartScreen; Fase D+
Total do 1º ano (3 lojas)	cerca de US$ 124	Somente Play + Apple; Microsoft não cobra

Tabela 6 — Custos e contas do plano de instalação nativa nos três sistemas.

Em calendário, o plano cabe em cerca de **cinco a oito semanas de esforço total**, das quais apenas uma depende de trabalho no motor: uma semana para a Fase A (adaptações de voz e armazenamento), duas para a Fase B (scaffold iOS e CI, incluindo validação em aparelho por TestFlight se a conta já existir), e duas a quatro de calendário para a Fase C (conta Apple, preenchimento de App Store Connect e ciclo de revisão — o tempo da Apple, não o nosso). Android e Windows não consomem semanas novas neste plano: já estão em produção pela Fase 1 do estudo anterior.

9. Veredito e Esforço por Rota

A Figura 1 atualiza o mapa de esforço do estudo I com as rotas iOS. As barras escuras marcam o que **já está implementado** (TWA Android e Electron Windows) — a expansão pedida se resume à barra do Capacitor iOS e às adaptações do motor. As rotas de reescrita (Swift nativo, Flutter multiplataforma) permanecem ordens de magnitude mais caras sem qualquer ganho para o usuário final, e seguem descartadas.

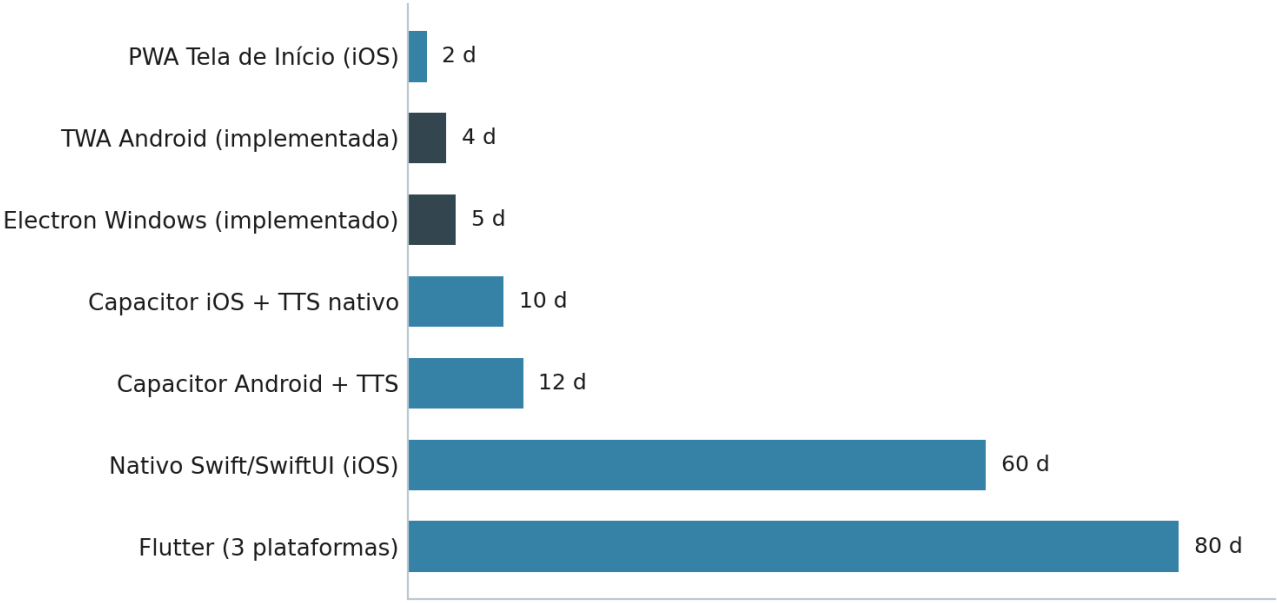


Figura 1 — Esforço estimado por rota, em dias de desenvolvimento (estimativas deste estudo II; barras escuras = já implementadas na Fase 1).

O veredito consolidado: **Android VIÁVEL (TWA, em produção) · iOS VIÁVEL (Capacitor com voz e armazenamento nativos, adaptações de 4-5 dias) · Windows VIÁVEL (Electron, em produção)**. A recomendação operacional em uma frase: manter a TWA e o Electron como estão, preparar o motor com os dois adaptadores (Fase A), criar o projeto Capacitor iOS com CI macOS (Fase B) e abrir a conta Apple para TestFlight e App Store quando o iOS virar prioridade de publicação (Fase C).

Plano em uma frase: TWA Android + Electron Windows mantidos + Capacitor iOS com voz e armazenamento nativos, tudo compilado no GitHub Actions

10. Plano de Ação em Fases

O plano de ação está dividido em quatro fases com entregáveis verificáveis e dependências explícitas. A **Fase A é executável imediatamente**, neste ambiente e no repositório da Klivfy, sem conta nova e sem custo — ela deixa o motor pronto para qualquer casco nativo e já melhora a robustez da voz no iOS web (o cancel() preventivo). A Fase B também não exige dinheiro, apenas o scaffold e o CI; só a Fase C depende da decisão comercial de pagar a taxa anual da Apple. As fases podem ser pausadas entre si sem retrabalho.

Fase	Escopo	Entregáveis verificáveis	Custo
Fase A — Motor bimodal (1 semana)	Adaptador de voz (cancel() preventivo + back-end nativo via plugin quando em Capacitor) e adaptador de armazenamento (Preferences quando em Capacitor); regressão completa nas 5 plataformas atuais	Engine v2.7.0 validado; QA em Chrome, Firefox, PWA, TWA e Electron	R$ 0
Fase B — Scaffold iOS + CI (2 semanas)	Projeto Capacitor iOS no repositório (appId com.klivfy.contacomigo), ícone/splash, assets embutidos, workflow macOS no Actions gerando IPA (não assinado) como artefato	IPA de teste na aba Actions; instruções de sideload via Xcode/TestFlight	R$ 0
Fase C — Conta Apple + lojas (2-4 semanas de calendário)	Apple Developer Program; assinatura via secrets; TestFlight com famílias beta; App Store Connect (Kids 5-, labels, metadados); publicar também Play/Store se pendentes	App aprovado na App Store; builds beta via TestFlight	US$ 99/ano
Fase D — Evolução (contínua)	Notificações locais dos baús de ofensiva, atalhos e widgets opcionais; unificação Capacitor Android se algum dia valer a pena; domínio próprio	Recursos nativos incrementais por release	Opcional

Tabela 7 — Plano de ação em fases com entregáveis e custos.

11. Riscos e Mitigações

Nenhum risco identificado compromete a viabilidade; todos têm mitigação conhecida, e os dois de maior impacto (voz e armazenamento no iOS) são tratados estruturalmente na Fase A, antes mesmo do scaffold existir. O risco regulatório mais citado — a diretriz 4.2 da Apple — tem defesa forte porque o jogo roda localmente com motor próprio e ganha capacidades nativas reais no casco. Os riscos operacionais (ciclo de revisão, dependência de Mac, manutenção de mais um wrapper) são típicos de publicação e absorvidos pelo CI.

Risco	Probabilidade	Impacto	Mitigação
Voz com particularidades no iOS (fila silenciosa, vozes variáveis entre versões)	Média	Alto	Fase A: cancel() antes de cada speak() + segundo back-end nativo (AVSpeechSynthesizer) com detecção automática
Evicção de localStorage no WKWebView	Média	Alto	Fase A: adaptador Preferences (nativo, sem evicção); backup 1-clique do painel dos responsáveis como cinta
Rejeição pela diretriz 4.2 (funcionalidade mínima)	Baixa-média	Alto	Jogo roda local com motor próprio e assets embutidos; capacidades nativas reais; TestFlight antes da submissão
Ciclo de revisão da Apple com rejeições iterativas	Média	Baixo	Metadados completos, labels de privacidade simples (zero coleta); iteração por TestFlight; Play/Store não dependem disso
Dependência de Mac para compilar/assinar iOS	Certa (sem CI)	Baixo	Build no GitHub Actions com executor macOS (grátis em repo público); assinatura via secrets
Manutenção de mais um wrapper nativo	Média	Baixo	Fonte única do jogo; wrappers finos versionados e compilados no CI; TWA/Capacitor/Electron desacoplados
Taxa anual da Apple em caso de pausa do projeto	Certa	Baixo	iOS entra em produção só na Fase C; até lá, web app da Tela de Início mantém famílias iOS atendidas sem custo

Tabela 8 — Riscos identificados e mitigações (estudo II).

12. Conclusão

O pedido de instalação nativa nos três sistemas é **viável, de custo controlado e sem reescrita do jogo**. A assimetria entre os ecossistemas define o desenho: no Android e no Windows a instalação nativa já existe (TWA e Electron, produzidos pelo CI desde a Fase 1); no iOS, o único caminho em escala é a App Store, e o Capacitor o alcança com o motor intacto, ao preço de duas adaptações pequenas e bem delimitadas — a voz, que ganha um segundo back-end nativo, e o armazenamento, que ganha persistência real pela API Preferences. Ambas melhoram o motor em todas as plataformas, e nenhuma introduz dependência nova de runtime.

A recomendação imediata é aprovar a execução da **Fase A**, que é gratuita, cabe em uma semana e deixa o motor v2.7.0 pronto para qualquer casco nativo — inclusive com a voz mais robusta no iOS web já hoje. Em seguida, a Fase B materializa o projeto iOS e o pipeline macOS, entregando um IPA de teste sem qualquer gasto; a Fase C fica condicionada à decisão comercial de contratar o Apple Developer Program (US$ 99/ano), com o atalho gratuito da Tela de Início cobrindo famílias iOS enquanto essa decisão amadurece. Ao fim da Fase C, o Conta Comigo estará instalável como aplicativo nativo nos três grandes sistemas, com a mesma base de código e o mesmo compromisso de privacidade que o caracteriza desde a origem.